# Use Apache Spark in Microsoft Fabric

## Overview of Apache Spark in Fabric

- **Definition**: Spark is an open-source, parallel processing framework designed for large-scale data analytics.
- **Distributed Architecture**: It uses a "divide and conquer" approach, distributing tasks across a **head node** (driver program) and multiple **worker nodes** (executors).
- **Integration**: In Microsoft Fabric, Spark enables the processing and analysis of data directly within a **Lakehouse** at scale.

## Configuration and Environments

- **Spark Pools**: Workspaces include a "starter pool" for quick setup, but you can create custom pools to optimize nodes based on workload needs or cost.
- **Custom Environments**: You can define specific Spark runtimes, install public (PyPI) or custom libraries, and configure Spark properties to ensure consistent execution environments.
- **Optimization Options**:
    - **Native Execution Engine**: A vectorized engine that improves performance for Parquet and Delta files.
    - **High Concurrency Mode**: Allows multiple users or processes to share a single Spark session efficiently while maintaining code isolation.

## Running Spark Code

- **Notebooks**: Used for interactive data exploration and collaboration. They support multiple languages (PySpark, Scala, Spark SQL, Spark R) and combine code with markdown text.
- **Spark Job Definitions**: Used for automated, non-interactive processes. These allow you to run scripts on-demand or on a set schedule.

## Data Manipulation and Visualization

- **Dataframes**: The primary structure for working with structured data. They allow for:
    - **Schema Inference**: Automatically detecting column names and types, though explicitly defining a schema improves performance.
    - **Transformations**: Methods to filter, sort, group, and aggregate data.
    - **Partitioning**: Saving data into folder hierarchies (e.g., Category=Road Bikes) to maximize performance and reduce disk I/O.
- **Spark SQL**: Enables querying data using SQL expressions. You can create **temporary views** for session-level analysis or **managed tables** (stored in Delta format) for persistence.
- **Visualization**:
    - **Built-in Charts**: Notebooks offer native UI tools to quickly convert table results into visual charts.
    - **Graphics Libraries**: For more control, users can use Python libraries like **Matplotlib** or **Seaborn** to create highly customized visualizations.

## Questions

1. What is the expected result of executing the following PySpark command:
df.groupBy('Category').agg({'SalesAmount': 'sum'})?
A. A list of categories with their corresponding average sales amounts.
B. A new dataframe with each category and the sum of sales amounts for each category.
C. A summary table showing the count of items in each category.
D. A modified version of the original dataframe with a new total column.

2. When creating a custom environment in Microsoft Fabric, which setting allows you to use a specific version of Apache Spark for your tasks?
A. Enable high concurrency mode in the workspace settings.
B. Choose the node family type in the Spark pool configuration.
C. Specify the Spark runtime in the environment settings.
D. Upload a custom resource file to the environment.

3. You have a dataframe with a column named 'Date' in a string format, and you need to convert it to a date format for further analysis. Which PySpark method would you use?
A. filter.
B. selectExpr.
C. withColumn.
D. groupBy.

4. You have a dataframe containing sales data and want to quickly visualize the total sales per category using built-in functionality in Microsoft Fabric notebooks. Which feature should you use?
A. Matplotlib library.
B. Seaborn library.
C. Built-in notebook charts.
D. Power BI Desktop integration.

5. When working with large datasets, which Spark Dataframe method is preferred for performance when only a subset of columns is needed?
A. groupBy.
B. select.
C. filter.
D. agg.

6. Which practice should be followed to optimize data processing performance in a Spark pool using the native execution engine in Microsoft Fabric?
A. Set the Spark pool to use CPU optimized nodes exclusively.
B. Disable dynamic allocation to ensure consistent resource allocation.
C. Enable the native execution engine at the environment level by setting relevant Spark properties.
D. Use only CSV files for all data storage.

7. Your organization is using Spark SQL to analyze transaction data and needs to generate time-series visualizations. What should be considered when choosing a visualization method?
A. The ease of exporting visualizations to PowerPoint.
B. The ability to handle large volumes of data efficiently.
C. The availability of built-in templates for time-series data.
D. Whether the library supports Java and Scala.

8. What configuration should be set in a custom environment to ensure that specific Python libraries are available for a Spark job in Microsoft Fabric?
A. Select the high concurrency mode in the Spark settings.
B. Configure the Spark runtime to the latest version available.
C. Install specific public libraries from the Python Package Index (PyPI) in the environment settings.
D. Upload the libraries as resource files to the Lakehouse.

9. A data scientist wants to compare product performance across different categories using Spark SQL. What is an efficient approach to accomplish this task?
A. Use Excel for data comparison, then import results into a notebook.
B. Analyze product data using Python scripts outside Microsoft Fabric.
C. Use Spark SQL to perform group-by queries on product data and visualize results using notebook charts.
D. Manually count records using the Lakehouse explorer.

10. Which node in a Spark pool is responsible for coordinating distributed processes through a driver program?
A. The worker node.
B. The head node.
C. The executor node.
D. The storage node.

11. When saving a dataframe, which file format is typically preferred in Microsoft Fabric for large-scale analytical systems due to its efficiency?
A. CSV.
B. JSON.
C. Parquet (or Delta).
D. XML.

12. To improve performance by eliminating unnecessary disk IO when filtering queries, which technique should be used when writing a dataframe to storage?
A. Caching.
B. Partitioning (using the partitionBy method).
C. Indexing.
D. Compression.

13. What is the primary difference between a managed table and an external table in the Spark catalog regarding data deletion?
A. Deleting an external table deletes the underlying data, while deleting a managed table does not.
B. Deleting a managed table deletes the underlying data; deleting an external table does not.
C. There is no difference; both delete the underlying data.
D. Managed tables cannot be deleted once created.

14. Which "magic" command is used at the top of a notebook cell to specifically run Scala code?
A. %%pyspark.
B. %%sql.
C. %%spark.
D. %%configure.

15. If you need to share a Spark session across multiple concurrent users to optimize resource usage, which mode should you enable?
A. Autoscale.
B. High concurrency mode.
C. Dynamic allocation.
D. Standard session.

16. Which specific Spark property must be set to true to enable the native execution engine at the environment level?
A. spark.shuffle.manager.
B. spark.native.enabled.
C. spark.sql.execution.native.
D. spark.vectorized.engine.

17. What happens to a temporary view created with createOrReplaceTempView when the Spark session ends?
A. It is persisted in the Spark catalog.
B. It is converted into a managed table.
C. It is automatically deleted.
D. It is saved as a Parquet file in OneLake.

18. Which Python library, often used for custom visualizations, requires a Spark dataframe to be converted to a Pandas dataframe using the toPandas() method?
A. Seaborn.
B. PySpark.sql.functions.
C. Matplotlib.
D. Delta Lake.

19. In a Spark pool, what is the role of the worker nodes?
A. To coordinate distributed processes through a driver program.
B. To perform the actual data processing tasks via executor processes.
C. To store the Spark runtime configuration.
D. To manage user authentication and workspace permissions.

## Answers

1. B
2. C
3. C
4. C
5. B
6. C
7. B
8. C
9. C
10. B
11. C
12. B
13. B
14. C
15. B
16. B
17. C
18. C
19. B